Event Venue Finder for Münster

A Web-Based GIS Application

Khanh Giang Le 556087
Lerissa Merril Menezes 564864

Project Overview

🎯 Goal

An interactive map for discovering event venues in Münster with public transport accessibility.

📦 Datasets

Self-compiled event venues + external Münster bus stop locations, both in GeoJSON.

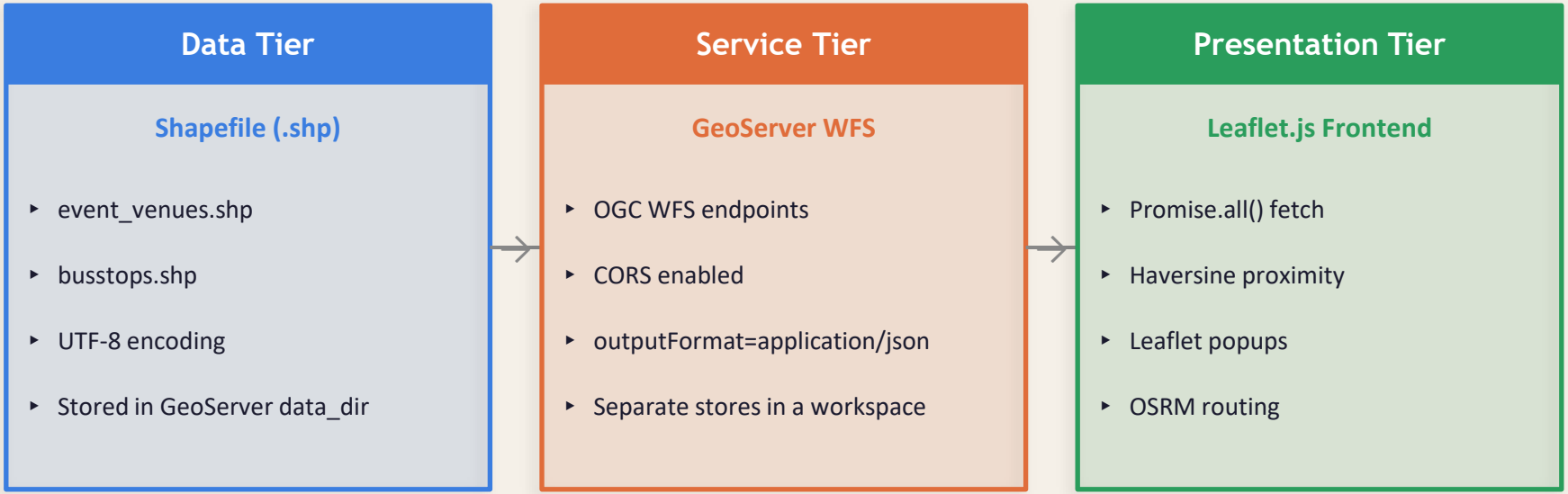
⚙️ Stack

GeoServer (WFS) · Leaflet.js · OSRM Routing · HTML / CSS / JavaScript

🗺️ Key Output

Venue popups with full attributes, nearest bus stop, and walking route on click.

System Architecture



⚠️ CORS required: CrossOriginFilter enabled in web.xml with chainPreflight=false for Jetty 10 compatibility

Data Preparation

01

Author GeoJSON

Self-compiled venue dataset with attributes. External bus stop GeoJSON for Münster.

02

Convert in QGIS

Export to Shapefile with UTF-8 encoding. Add .cpg file to fix umlaut rendering.

03

Publish in GeoServer

Create workspace → store → publish layer. Compute bounding box from data.

04

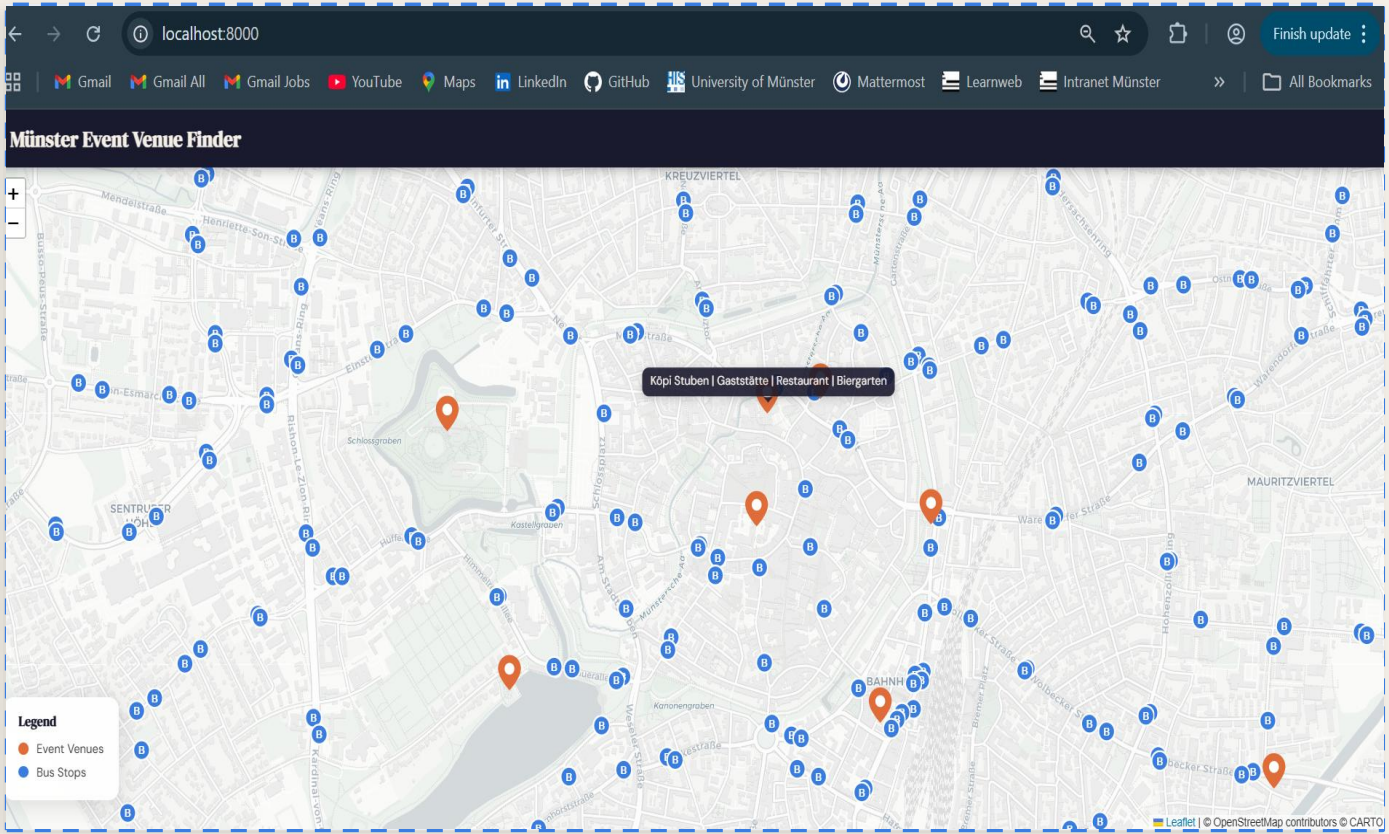
Verify WFS

Test endpoint in browser. Confirm GeoJSON response with correct features.

⚠ Shapefile Limitations

Field names	Truncated to 10 chars e.g. available_evenings → available_
Arrays	Flattened to comma-separated strings
Encoding	Requires explicit UTF-8 + .cpg companion file
Booleans	Stored as 0/1 integers, not true/false

Map Interface



Orange markers
Event venues

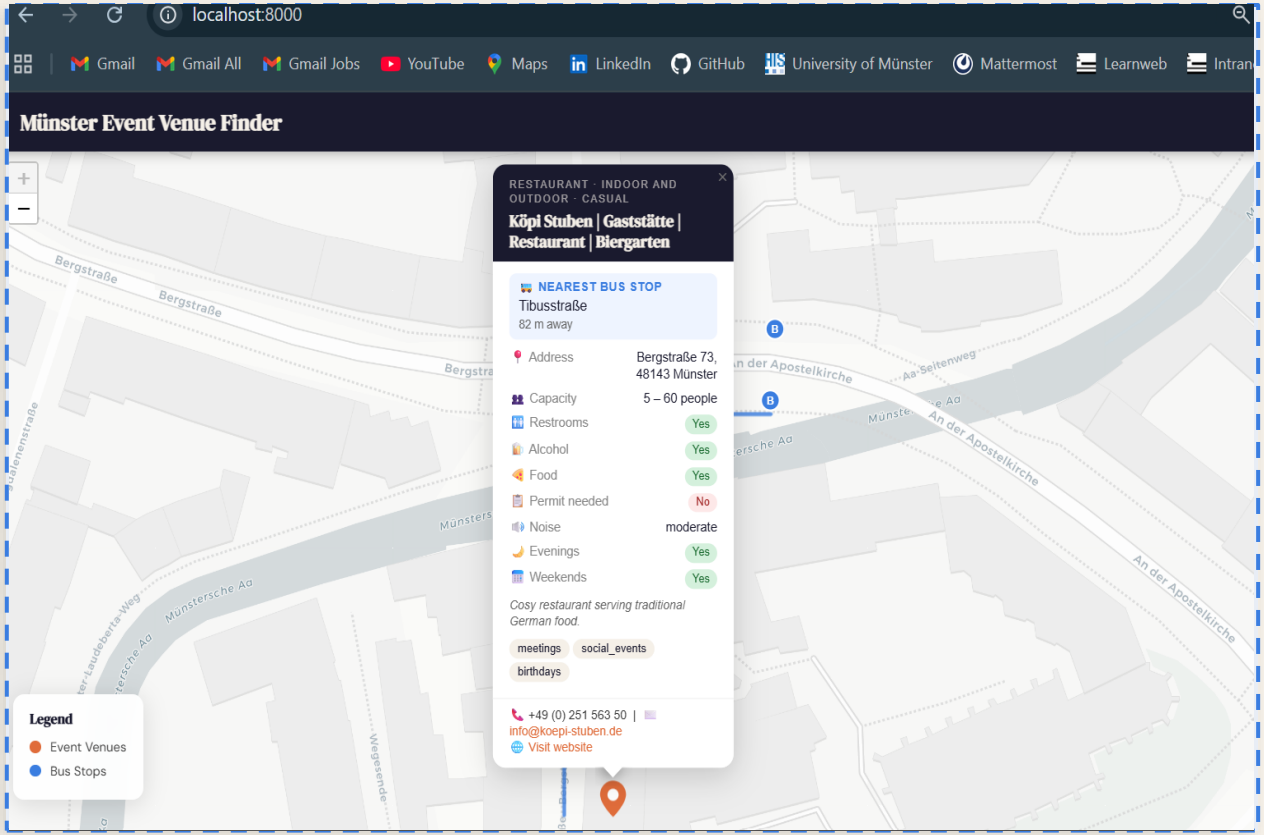
Blue markers
Bus stops

Hover tooltip
Shows venue/stop name

Click venue
Opens full popup

Click venue
Draws walking route

Venue Popup & Nearest Bus Stop

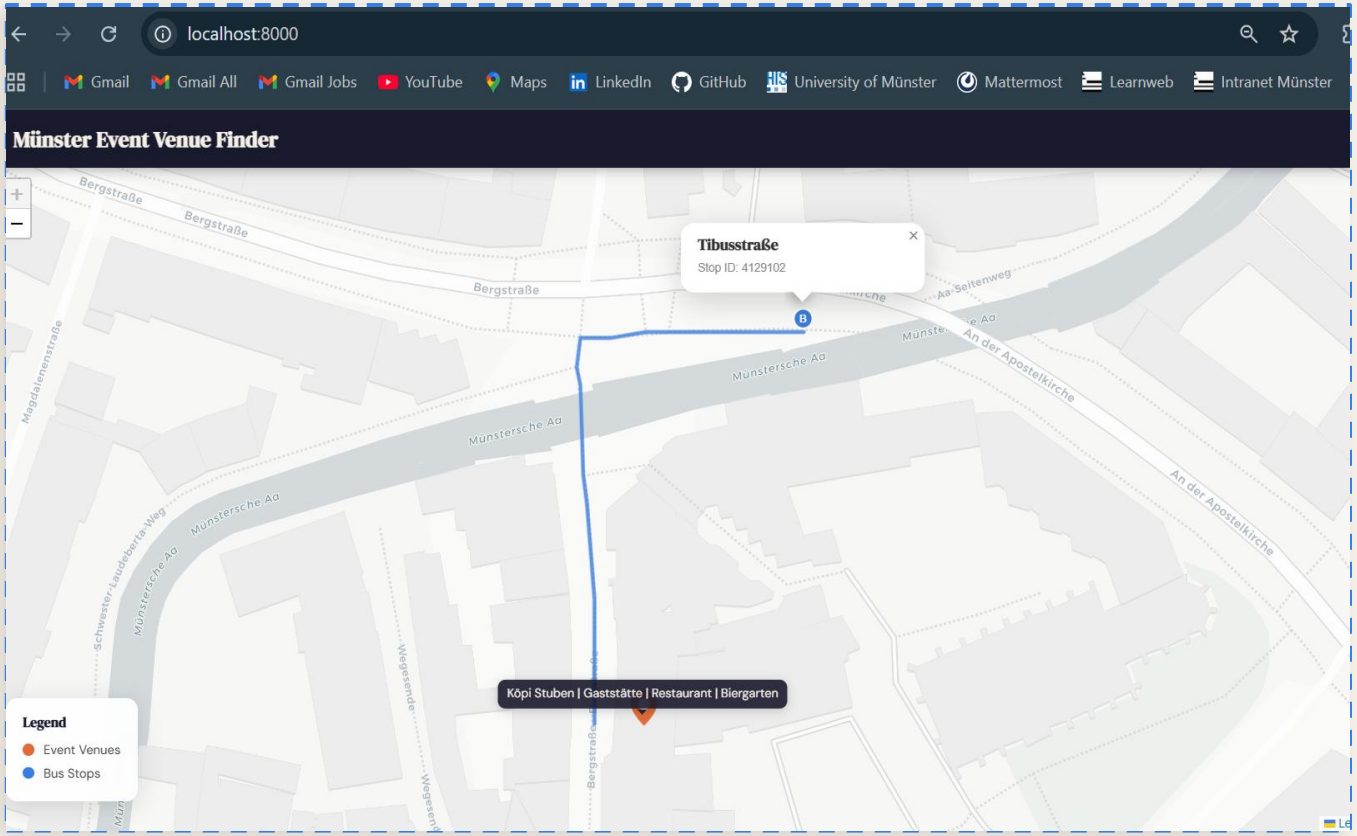


Popup Attributes

🚌 **Nearest Bus Stop → Calculated via Haversine formula**

- 📍 Address
- 👥 Min / Max Capacity
- 🚻 Restrooms
- 🍺 Alcohol allowed
- 🍕 Food allowed
- 📋 Permit required
- 🔊 Noise restrictions
- 🌙 Available evenings
- 📅 Available weekends

Walking Route to Nearest Bus Stop



1. **User clicks venue**
   layer.on('click') event fires
2. **Find nearest stop**
   Haversine loop over all bus stops
3. **Call OSRM**
   Pedestrian routing via routing.openstreetmap.de
4. **Draw route**
   Blue polyline, previous route removed

Challenges & Lessons Learned

CORS Configuration

web.xml CrossOriginFilter with chainPreflight=false required for Jetty 10. Both filter and filter-mapping blocks needed.

Shapefile Field Truncation

All attribute references in map.js had to match 10-char truncated names from GeoServer (e.g. available_ vs available_evenings).

UTF-8 Encoding

German umlauts corrupted without explicit UTF-8 export in QGIS and a .cpg companion file in the shapefile folder.

Client-Side Spatial Logic

Haversine formula computed entirely in the browser — no server-side spatial query needed for nearest stop calculation.

Summary

- GeoJSON datasets published through GeoServer as OGC WFS services
- Leaflet.js frontend fetches both layers in parallel via Promise.all()
- Nearest bus stop computed client-side using the Haversine formula
- Walking route drawn on click using Leaflet Routing Machine + OSRM
- Shapefile limitations required workarounds for field names, arrays, and encoding